Monoids: an introduction to lawful structures

George Wilson

george@wils.online

13th October 2026

Outline

Let's look at:

- Basic Haskell syntax
- A simple example type class
- The Monoid type class (including laws)
- Examples of monoid instances
- Examples of functions defined for all monoids
- Why laws are helpful

Values and types

```
x :: Int
x = 5 + 10
```

Strings and concatenation

```
bfpg :: String
bfpg = "BFPG"

greeting :: String
greeting = "Hello, " ++ bfpg ++ "!"
```

Strings and concatenation

```haskell
bfpg :: String
bfpg = "BFPG"

greeting :: String
greeting = "Hello, " ++ bfpg ++ "!"
```

```
> greeting
-- "Hello, BFPG!"
```

Functions on strings

```
greet :: String -> String
greet name = "Hello, " ++ name ++ "!"
```

Functions on strings

```haskell
greet :: String -> String
greet name = "Hello, " ++ name ++ "!"
```

```haskell
> greet bfpg
-- "Hello, BFPG!"
```

Custom datatypes

```
data MeetupGroup =
  MeetupGroup {
    name :: String,
    url :: String,
    description :: String
  }
```

Custom datatypes

```haskell
data MeetupGroup =
  MeetupGroup {
    name :: String,
    url :: String,
    description :: String
  }
bfpgMeetup :: MeetupGroup
bfpgMeetup =
  MeetupGroup {
    name = "Brisbane Functional Programming Group",
    url = "https://luma.com/bfpg",
    description = "a bunch of lambda heads"
  }
```

Custom data types

```
data TrafficLight
  = Red
  | Yellow
  | Green
```

Type classes

```
class Show a where
  show :: a -> String
```

Type classes

```
class Show a where
  show :: a -> String

instance Show TrafficLight where
  show tl =
    case tl of
      Red -> "Red"
      Yellow -> "Yellow"
      Green -> "Green"
```

Functions with type class constraints

```
greet :: Show a => a -> String
greet s = "Hello, " ++ show s ++ "!"
```

Functions with type class constraints

```
greet :: Show a => a -> String
greet s = "Hello, " ++ show s ++ "!"
```

```
> greet Yellow
-- "Hello, Yellow!"
```

Functions with type class constraints

```
greet :: Show a => a -> String
greet s = "Hello, " ++ show s ++ "!"
```

```
> greet Yellow
-- "Hello, Yellow!"

> greet 5
-- "Hello, 5!"
```

Functions with type class constraints

```
greet :: Show a => a -> String
greet s = "Hello, " ++ show s ++ "!"
```

```
> greet Yellow
-- "Hello, Yellow!"

> greet 5
-- "Hello, 5!"

> greet (++)
-- Error: No Show instance for
--          (String -> String -> String)
```

Monoid

```
class Monoid m where
  mappend :: m -> m -> m
  mempty  :: m
```

Monoid

```haskell
class Monoid m where
  mappend :: m -> m -> m
  mempty  :: m

-- mappend as an infix operator, for convenience
x <> y = mappend x y
```

Monoid

```
class Monoid m where
  mappend :: m -> m -> m
  mempty  :: m

-- mappend as an infix operator, for convenience
x <> y = mappend x y

instance Monoid String where
  mappend s t = s ++ t
  mempty = ""
```

Monoid laws

Left identity:

```
-- for all x.
mempty <> x == x
```

Monoid laws

Left identity:

```
-- for all x.
mempty <> x == x
```

Right identity:

```
-- for all x.
x <> mempty == x
```

Monoid laws

Left identity:

```
-- for all x.
mempty <> x == x
```

Right identity:

```
-- for all x.
x <> mempty == x
```

Associativity:

```
-- for all x y z.
x <> (y <> z) == (x <> y) <> z
```

Monoid laws specialised to strings

Left identity:

```
-- for all x.
"" ++ x == x
```

Right identity:

```
-- for all x.
x ++ "" == x
```

Associativity:

```
-- for all x y z.
x ++ (y ++ z) == (x ++ y) ++ z
```

What are some other types that fit monoid?

Integer instances?

```
-- integers?
x + (y + z) == (x + y) + z
```

Integer instances?

```
-- integers?
x + (y + z) == (x + y) + z
x * (y * z) == (x * y) * z
```

Integer instances?

We can't have multiple instances like this in Haskell

```
instance Monoid Integer where
  mappend x y = x + y
  mempty = 0

instance Monoid Integer where
  mappend x y = x * y
  mempty = 1
```

Integer instances with newtype wrappers

```haskell
newtype Sum = Sum { unwrapSum :: Integer }
```

Integer instances with newtype wrappers

```haskell
newtype Sum = Sum { unwrapSum :: Integer }

instance Monoid Sum where
  mappend (Sum x) (Sum y) = Sum (x + y)
  mempty = Sum 0
```

Integer instances with newtype wrappers

```haskell
newtype Sum = Sum { unwrapSum :: Integer }
instance Monoid Sum where
  mappend (Sum x) (Sum y) = Sum (x + y)
  mempty = Sum 0

newtype Product = Product { unwrapProduct :: Integer }
instance Monoid Product where
  mappend (Product x) (Product y) = Product (x * y)
  mempty = Product 1
```

Derived functions

```
mconcat :: Monoid m => [m] -> m
```

```
> mconcat ["Hello", ", ", "BFPG", "!"]
-- "Hello, BFPG!"
```

Trees

```
data Tree a = Leaf a | Branch (Tree a) (Tree a)
```

Trees

```haskell
data Tree a = Leaf a | Branch (Tree a) (Tree a)

concatTree :: Monoid m => Tree m -> m
concatTree t =
  case t of
    Leaf m -> m
    Branch t u -> concatTree t <> concatTree u
```

Derived functions

```
class Foldable f where
  fold :: Monoid m => f m -> m
```

Derived functions

```
class Foldable f where
  fold :: Monoid m => f m -> m
  -- much more exciting
  foldMap :: Monoid m => (a -> m) -> f a -> m
```

Derived functions

```
class Foldable f where
  fold :: Monoid m => f m -> m

  -- much more exciting
  foldMap :: Monoid m => (a -> m) -> f a -> m

instance Foldable [] where
  foldMap a2m list = mconcat (map a2m list)
```

Derived functions

```
class Foldable f where
  fold :: Monoid m => f m -> m

  -- much more exciting
  foldMap :: Monoid m => (a -> m) -> f a -> m

instance Foldable [] where
  foldMap a2m list = mconcat (map a2m list)

instance Foldable Tree where
  foldMap a2m tree =
    case tree of
      Leaf a -> a2m a
      Branch t u -> foldMap a2m t <> foldMap a2m u
```

Thanks for listening!